

Oblak

Tehnička dokumentacija projekta

Serverless platforma za izvršavanje Python koda

1. Uvod i pregled sistema

Oblak je serverless platforma za izvršavanje korisničkog Python koda na serveru. Ideja sistema je slična servisima kao što su AWS Lambda ili Google Cloud Functions: korisnik šalje Python funkciju, platforma je proverava, priprema okruženje za izvršavanje i omogućava pokretanje funkcije preko generisanog HTTP endpoint-a.

Najvažnija bezbednosna pretpostavka sistema je da je korisnički kod nepouzdan. Kod koji korisnik uploaduje može biti benigni, ali može biti i maliciozan, pa ga sistem tretira kao potencijalno opasan unos. Zbog toga platforma koristi više slojeva zaštite: autentifikaciju, validaciju ulaza, statičku analizu, Docker sandbox, ograničenja resursa i audit logovanje.

Specifikacija projekta preporučuje Firecracker MicroVM za izolaciju izvršavanja. U ovoj implementaciji se umesto Firecracker-a koristi Docker sandbox, jer je Firecracker znatno teži kao konfiguraciju na Windows okruženju. Firecracker je dokumentovan kao otvorena stavka.

1.1 Tok rada korisnika

Tipičan tok rada izgleda ovako:

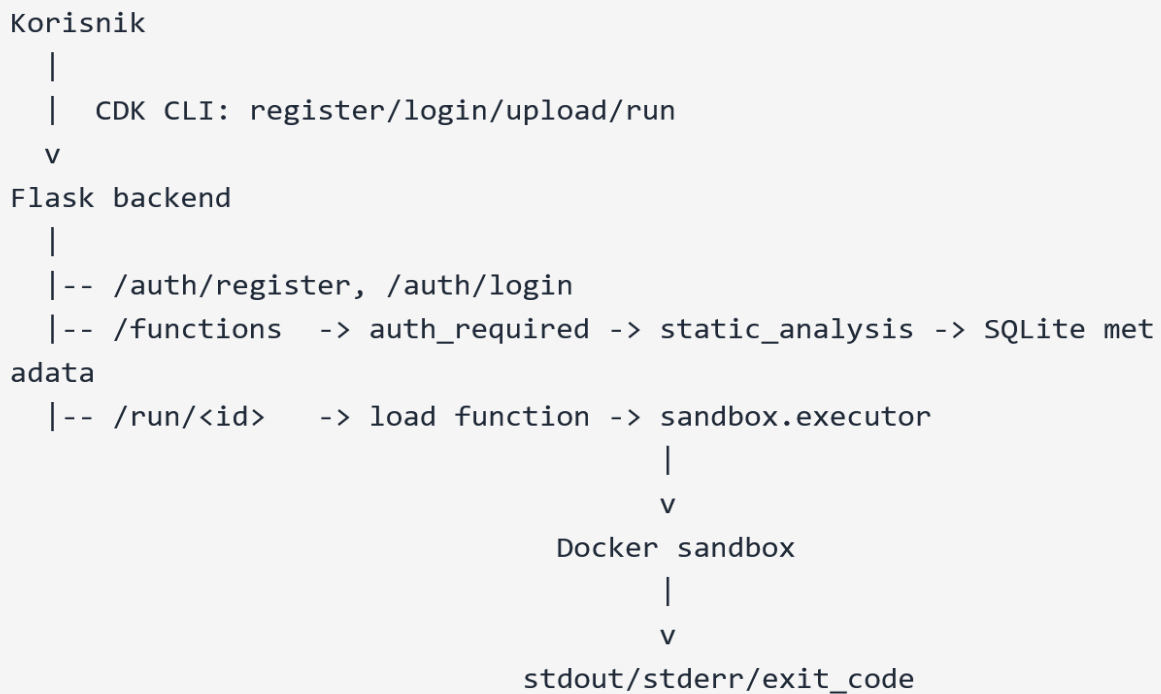
1. Korisnik se registruje ili prijavljuje pomoću CDK CLI aplikacije.
2. Server vraća API token.
3. CLI token čuva u `.oblak_token` fajlu ili ga čita iz `OBLAK_TOKEN` environment promenljive.
4. Korisnik uploaduje Python fajl i opciono `requirements.txt`.
5. Server privremeno čuva fajlove u `backend/storage/uploads/<uuid>/`.
6. Uploadovani Python fajl se analizira pomoću statičke AST analize iz `sandbox/static_analysis.py`.
7. Ako analiza prođe, funkcija dobija status `ready`; ako ne prođe, dobija status `rejected`, a fajlovi se brišu sa diska.
8. Za prihvaćenu funkciju server vraća `run_url` oblika `/run/<function_id>`.
9. Pozivom `run_url` endpoint-a server pokreće funkciju kroz Docker sandbox.
10. Rezultat izvršavanja se vraća korisniku kroz JSON odgovor.

1.2 Glavne komponente

Sistem je podeljen na sledeće komponente:

Komponenta	Lokacija	Odgovornost
Flask backend	backend/	Autentifikacija, upload, skladištenje metapodataka, run endpoint
CDK CLI	cli/	Konzolni klijent za register, login, upload i run
Sandbox	sandbox/	Statička analiza, Docker izvršavanje, limiti, sandbox audit
SQLite baza	backend/storage/ oblak.db	Korisnici i metapodaci funkcija
Docker konfiguracija	docker/	Dockerfile i compose konfiguracija
Skripte	scripts/	Setup i pokretanje projekta
Dokumentacija	docs/	Tehnička dokumentacija, STRIDE, testovi, audit

1.3 Arhitektura toka izvršavanja



2. Implementirane funkcionalnosti

2.1 Autentifikacija CDK CLI aplikacije

CDK CLI je implementiran u `cli/cdk.py`. Podržava komande:

```
python cli/cdk.py register <username> <password>
python cli/cdk.py login <username> <password>
python cli/cdk.py upload <name> <file.py> [--requirements requirements.txt]
python cli/cdk.py run <function_id> [--payload "{}"]
```

Autentifikacija koristi Bearer token. Token se generiše na backendu pomoću `secrets.token_urlsafe(32)`, čuva se u bazi i vraća korisniku pri registraciji ili login-u. CLI ga lokalno čuva u `.oblak_token` fajlu.

Lozinke se ne čuvaju u čistom tekstu. Implementirano je hešovanje pomoću PBKDF2-HMAC-SHA256 sa nasumičnom soli i 200.000 iteracija.

2.2 Upload Python koda

Upload je implementiran u `backend/routes/upload.py`. Ruta `POST /functions` je zaštićena pomoću `auth_required` dekoratora.

Upload prihvata:

- `code`: Python fajl,
- `requirements`: opcioni `requirements.txt`,
- `name`: ime funkcije,
- `entrypoint`: naziv glavnog Python fajla.

Nazivi fajlova se normalizuju pomoću `secure_filename()`. Podržan je upload jednog `.py` fajla i opcionog `requirements.txt` fajla.

Važna implementaciona napomena: kod se prvo privremeno sačuva na disk, zatim se analizira. Ako analiza ne prođe, direktorijum funkcije se briše pomoću `shutil.rmtree()`. Dakle, analiza se radi pre označavanja funkcije kao `ready`, ali ne pre svakog pisanja na disk.

2.3 Statička analiza koda

Statička analiza je implementirana u `sandbox/static_analysis.py`. Backend ne koristi poseban `backend/analysis.py`, već direktno koristi analizu iz `sandbox` dela.

Analiza koristi Python `ast` modul. Kod se parsira kao AST stablo, bez izvršavanja.

Zabranjeni moduli:

```
os, subprocess, sys, socket, requests, urllib, shutil, pathlib,
glob, ctypes, multiprocessing, threading
```

Zabranjene ugrađene funkcije:

```
eval, exec, compile, open, __import__, input
```

Statička analiza se koristi u dva trenutka:

1. Pri upload-u funkcije.
2. Neposredno pre izvršavanja u `sandbox/executor.py`.

Ovo predstavlja odbranu u dubinu: čak i ako se fajl promeni nakon upload-a, izvršavanje ponovo proverava kod.

Ograničenje: AST analiza detektuje direktne import-e i direktne pozive zabranjenih funkcija, ali nije potpuna zaštita od svih mogućih evasion tehnika. Zbog toga je Docker sandbox druga linija odbrane.

2.4 Priprema izvršnog okruženja i `requirements.txt`

Priprema izvršnog okruženja je implementirana u `sandbox/docker_runner.py`.

Ako funkcijski direktorijum sadrži `requirements.txt`, runner pravi poseban Docker image:

1. Izračunava SHA-256 hash sadržaja `requirements.txt`.
2. Pravi tag oblika `oblak-function:<hash>`.
3. Gradi image na osnovu `python:3.10-slim`.
4. Pokreće `pip install --no-cache-dir -r /tmp/requirements.txt`.
5. Koristi taj image za izvršavanje funkcije.

Na ovaj način funkcije sa istim `requirements.txt` mogu ponovo koristiti isti image tag.

Napomena: sadržaj `requirements.txt` se trenutno ne bezbednosno skenira. To je otvorena stavka, jer dependency supply-chain napadi nisu rešeni samo instalacijom paketa u Docker image.

2.5 Run URL i izvršavanje funkcije

Za svaku prihvaćenu funkciju backend vraća `run_url`, na primer:

```
http://127.0.0.1:5000/run/<function_id>
```

Ruta `POST /run/<function_id>` trenutno je javno dostupna. To znači da svako ko zna UUID funkcije može pokrenuti funkciju. Ovo je svesna otvorena stavka i dokumentovana je u threat model-u.

Endpoint vraća:

- `stdout`,
- `stderr`,
- `return_code`,
- `sandbox metadata`.

Status kodovi:

Situacija	HTTP status
Uspešno izvršavanje	200
Funkcija nije pronađena	404
Funkcija nije <code>ready</code>	409
Blokirano sandbox politikom	403
Docker/sandbox nije dostupan	503
Timeout	504
Runtime greška	500

2.6 Docker sandbox

Korisnički kod se izvršava kroz Docker kontejner. Docker komanda koristi sledeća ograničenja:

Mehanizam	Implementacija	Svrha
Bez mreže	<code>--network none</code>	Sprečava mrežnu komunikaciju i eksfiltraciju

Mehanizam	Implementacija	Svrha
Memorijski limit	<code>--memory 128m</code>	Sprečava iscrpljivanje RAM-a
CPU limit	<code>--cpus 0.5</code>	Sprečava monopolizaciju CPU-a
PID limit	<code>--pids-limit 64</code>	Smanjuje rizik od fork bomb napada
Non-root korisnik	<code>--user 1000:1000</code>	Smanjuje privilegije u kontejneru
Read-only filesystem	<code>--read-only</code>	Sprečava pisanje po image filesystem-u
Ograničen /tmp	<code>--tmpfs /tmp:rw,noexec,nosuid,size=10m</code>	Omogućava minimalan privremeni prostor bez izvršavanja
Read-only mount koda	<code>-v <dir>:/app:ro</code>	Sprečava modifikaciju uploadovanog koda
Timeout	<code>TIMEOUT = 5</code>	Prekida beskonačne petlje

Docker nije jednako jak izolacioni mehanizam kao Firecracker MicroVM, ali za potrebe projekta pruža praktičan i testiran sandbox sloj, naročito u Windows razvojnom okruženju.

3. Bezbednosni zahtevi

ID	Kategorija	Zahtev	Implementacija
SR-01	Autentifikacija	Rute koje menjaju stanje sistema moraju zahtevati Bearer token	<code>auth_required</code> na <code>/functions</code>
SR-02	Hešovanje lozinki	Lozinke ne smeju biti čuvane u čistom tekstu	PBKDF2-HMAC-SHA256 u <code>models/user.py</code>
SR-03	Validacija ulaza	Korisnički nazivi fajlova moraju biti sanitizovani	<code>secure_filename()</code> u <code>upload.py</code>
SR-04	Ograničenje upload-a	Upload mora imati maksimalnu veličinu	<code>MAX_CONTENT_LENGTH = 1MB</code>
SR-05	Statička analiza	Python kod mora biti analiziran pre prihvatanja kao <code>ready</code>	<code>sandbox/static_analysis.py</code>
SR-06	Izolacija izvršavanja	Nepouzdan kod mora raditi u izolovanom okruženju	Docker sandbox
SR-07	Mrežna izolacija	Korisnički kod ne sme imati mrežni pristup	<code>--network none</code>

ID	Kategorija	Zahtev	Implementacija
SR-08	Ograničenje resursa	Kod ne sme iscrpeti CPU, memoriju ili procese	<code>--memory</code> , <code>--cpus</code> , <code>--pids-limit</code> , <code>timeout</code>
SR-09	Izolacija fajl sistema	Kod ne sme menjati host fajlove	Read-only mount i read-only container FS
SR-10	Audit	Bezbednosno relevantni događaji moraju biti zabeleženi	<code>sandbox/logs/audit.log</code>
SR-11	Odbrana u dubinu	Analiza se ponavlja pre izvršavanja	<code>executor.py</code> ponovo poziva statičku analizu

4. Model pretnji i STRIDE analiza

4.1 Pretpostavke

Sistem se analizira pod sledećim pretpostavkama:

- Korisnički Python kod je nepouzdan.
- Korisnik može pokušati da izvrši OS komande, mrežne konekcije, beskonačne petlje ili pristup fajl sistemu.
- Napadač može pokušati da ukrade ili pogodi API token.
- Napadač može pokušati da zloupotrebi `requirements.txt`.
- Lokalni fajl sistem i SQLite baza nisu kriptografski zaštićeni.
- Sistem je razvojni/fakultetski projekat, a ne produkcion cloud servis.

4.2 STRIDE tabela

ID	STRIDE	Komponenta	Pretinja	Uticaj	Postojeća mitigacija	Status
T-01	Spoofing	Auth/API token	Napadač koristi tuđi token	Neovlašćen upload	Token visoke entropije, Bearer provera	Delimično
T-02	Spoofing	Transport	Token može biti presretnut preko HTTP-a	Kompromitacija naloga	Nema HTTPS u razvojnoj konfiguraciji	Otvoreno
T-03	Tampering	Upload storage	Izmena fajla nakon analize	Izvršavanje izmenjenog koda	Ponovna analiza pre izvršavanja	Implementirano
T-04	Tampering	SQLite baza	Promena statusa iz rejected u ready	Zaobilaženje kontrole	Parametrizovani upiti sprečavaju SQLi, ali baza nije šifrovana	Delimično
T-05	Tampering	Audit log	Brisanje ili izmena logova	Gubitak forenzičkog traga	JSONL audit postoji, ali nema potpisa	Otvoreno

ID	STRIDE	Komponenta	Pretinja	Uticaj	Postojeća mitigacija	Status
T-06	Repudiation	Korisničke akcije	Korisnik poriče upload/run	Otežana odgovornost	Audit beleži event, timestamp, user_id gde postoji	Delimično
T-07	Information Disclosure	API greške	Curenje internih detalja kroz greške	Pomoć napadaču	Strukturirani JSON odgovori; debug mode je rizik	Delimično
T-08	Information Disclosure	Baza	API tokeni su u čistom tekstu	Krađa sesija ako baza procuri	Lozinke su hešovane, tokeni nisu	Otvoreno
T-09	Information Disclosure	stdout/stderr	Korisnički kod može vratiti informacije o runtime-u	Curenje detalja okruženja	Ne prosleđuju se host env promenljive; nema mreže	Delimično
T-10	DoS	Upload ruta	Mnogo upload zahteva	Iscrpljivanje servera	MAX_CONTENT_LENGTH, ali nema rate limit-a	Delimično

ID	STRIDE	Komponenta	Pretnja	Uticaj	Postojeća mitigacija	Status
T-11	DoS	Sandbox	Beskonačna petlja ili resource exhaustion	Usporavanje hosta	Timeout, CPU/memory/PID limit	Implementirano
T-12	DoS	Disk	Mnogo funkcija ili velik storage	Popunjavanje diska	Rejected funkcije se brišu; nema kvote	Otvoreno
T-13	Elevation of Privilege	Docker	Container escape	Kompromitacija hosta	Non-root, read-only, limiti; nema seccomp/AppArmor profila	Delimično
T-14	Elevation of Privilege	/run/ruta	Svako sa UUID-om može pokrenuti funkciju	Neovlašćen izvršavanje	UUID je nepredvidiv, ali ruta nije authovana	Otvoreno
T-15	Supply chain	requirements.txt	Maliciozan ili ranjiv paket	Izvršavanje tokom build-a	Build u Docker kontekstu; nema dependency scan-a	Otvoreno

4.3 Analiza po STRIDE kategorijama

Spoofing

Sistem koristi Bearer token autentifikaciju za upload funkcija. Tokeni se generišu pomoću `secrets.token_urlsafe(32)`, što daje visok nivo entropije. Najveća otvorena slabost je nedostatak HTTPS-a u podrazumevanoj razvojnoj konfiguraciji, jer se token može presresti na mreži ako se sistem koristi van lokalnog okruženja.

Tampering

Path traversal je ublažen pomoću `secure_filename()`. SQL injection je ublažen parametrizovanim SQL upitima. Izmena uploadovanog koda nakon inicijalne analize se dodatno ublažava ponovnom statičkom analizom neposredno pre izvršavanja.

SQLite baza i audit log fajl nisu kriptografski zaštićeni. Ako napadač dobije pristup fajl sistemu servera, može pokušati direktno menjanje baze ili logova.

Repudiation

Sistem ima centralizovan audit log u JSONL formatu. Beleže se registracija, login događaji, upload, sandbox izvršavanja, blokade, timeout i greške. Ipak, audit log ne sadrži IP adresu, User-Agent, kriptografski potpis niti udaljenu replikaciju, pa nije potpun forenzički dokaz.

Information Disclosure

Sistem ne vraća `storage_path` kroz `/functions` odgovor. Interna putanja se čuva u bazi i koristi interno. Potencijalni rizici ostaju Flask debug mode, previše detaljan `stderr`, i činjenica da API tokeni u bazi nisu hešovani.

Sandbox audit loguje samo `stderr_summary` do 200 karaktera, što smanjuje rizik od čuvanja velikih ili osetljivih izlaza u logu.

Denial of Service

Docker sandbox štiti od beskonačnih petlji i resource exhaustion napada kroz timeout, memorijski limit, CPU limit i PID limit. HTTP sloj ostaje ranjiviji deo, jer nema rate limiting-a po IP adresi ili korisniku. Takođe nema disk kvote po korisniku.

Elevation of Privilege

Docker smanjuje rizik pokretanjem koda kao neprivilegovan korisnik, bez mreže, sa read-only fajl sistemom i ograničenim resursima. Ipak, Docker kontejneri dele kernel sa hostom, pa container escape ostaje teorijski rizik. Firecracker bi bio jači izolacioni sloj.

5. Implementirane mitigacije i otvorene stavke

5.1 Implementirane mitigacije

Mehanizam	Status	Objašnjenje
Bearer token autentifikacija	Implementirano	Upload ruta zahteva validan token
PBKDF2 hešovanje lozinki	Implementirano	Lozinke nisu u čistom tekstu
Parametrizovani SQL upiti	Implementirano	Smanjuje rizik od SQL injection-a
Sanitizacija naziva fajlova	Implementirano	<code>secure_filename()</code>
Statička AST analiza	Implementirano	Blokira opasne import-e i funkcije
Ponovna analiza pre izvršavanja	Implementirano	Defense in depth
Docker sandbox	Implementirano	Izolacija procesa i fajl sistema
<code>--network none</code>	Implementirano	Nema mrežnog pristupa

Mehanizam	Status	Objašnjenje
CPU/memory/PID limiti	Implementirano	Smanjuju DoS rizik
Timeout	Implementirano	Prekida beskonačne petlje
JSONL audit log	Implementirano	Centralizovan audit trail
<code>requirements.txt</code> build	Implementirano	Instalacija zavisnosti u Docker image

5.2 Otvorene stavke

ID	Otvorena stavka	Opis
OI-01	Firecracker MicroVM	Zamena Docker sandbox-a jačom VM izolacijom
OI-02	HTTPS	TLS ispred Flask aplikacije, npr. nginx/Caddy reverse proxy
OI-03	Rate limiting	Ograničenje broja zahteva po IP adresi i tokenu
OI-04	Auth na <code>/run</code>	Trenutno je run endpoint javan ako korisnik zna UUID

ID	Otvorena stavka	Opis
OI-05	Kriptografski audit log	HMAC potpis ili Merkle-chain za tamper-evident logove
OI-06	Hešovanje API tokena	Čuvati samo hash tokena, slično lozinkama
OI-07	Dependency scan	<code>pip-audit</code> , <code>safety</code> ili allowlist paketa
OI-08	Seccomp/AppArmor	Dodatno ograničenje Docker sandbox syscall-a i fajl sistema
OI-09	Disk kvote	Limit po korisniku ili ukupni storage limit
OI-10	Produkcioni WSGI server	Umesto Flask debug servera koristiti Waitress/Gunicorn/uWSGI

6. Audit i revizija

Audit log je centralizovan u:

```
sandbox/logs/audit.log
```

Format je JSON Lines: svaka linija je jedan JSON objekat.

6.1 Backend audit događaji

Backend audit je implementiran u `backend/audit.py`. Koristi poseban logger `oblak.audit`, `FileHandler` i `propagate=False`, kako Flask/Werkzeug development logovi ne bi završavali u audit fajlu.

Backend loguje:

- `user_registered`,
- `login_success`,
- `login_failed`,
- `function_uploaded`,
- `function_executed`,
- `function_blocked`,
- `function_timeout`,
- `sandbox_unavailable`.

Primer:

```
{"event": "function_uploaded", "findings": [], "function_id": "6972750b-1c73-413b-be33-7a041baa17db", "status": "ready", "timestamp": "2026-05-30T21:36:29.093558+00:00", "user_id": 13}
```

6.2 Sandbox audit događaji

Sandbox audit je implementiran u `sandbox/executor.py`.

Sandbox loguje:

- `sandbox_execution_success`,
- `sandbox_execution_blocked`,
- `sandbox_execution_timeout`,
- `sandbox_execution_error`.

Primer uspešnog izvršavanja:

```
{"blocked_by_policy": false, "event": "sandbox_execution_success",  
"exit_code": 0, "reason": "", "script_path": "sandbox/test_scripts/  
benign/hello.py", "success": true, "timeout": false, "timestamp":  
"2026-05-30T21:41:54.701286+00:00"}
```

Primer timeout-a:

```
{"blocked_by_policy": false, "event": "sandbox_execution_timeout",  
"exit_code": 124, "reason": "Timeout", "script_path": "sandbox/test_  
_scripts/malicious/infinite_loop.py", "stderr_summary": "Execution  
stopped: timeout of 5 seconds exceeded", "success": false, "timeout  
": true, "timestamp": "2026-05-30T21:41:59.731295+00:00"}
```

Sistem ne loguje:

- lozinke,
- API tokene,
- ceo korisnički kod,
- ceo payload,
- ceo stdout,
- pun stderr ako je dugačak.

stderr se u sandbox audit-u skraćuje na `stderr_summary`.

7. Statička analiza projekta alatom Bandit

Bandit je instaliran i pokrenut nad projektom.

7.1 Komanda za produkcionni kod

```
bandit -r backend sandbox cli -x backend/storage,sandbox/logs,sandb  
ox/test_scripts
```

Rezultat:

```
Total lines of code: 807
Total issues by severity:
  High: 1
  Medium: 2
  Low: 4
```

7.2 Nalazi nad produkcionim kodom

ID	Severity	Fajl	Opis	Procena
B201	High	backend/app.py	Flask pokrenut sa debug=True	Stvaran nalaz za produkciju; prihvatljivo za razvoj, otvorena stavka za produkciju
B310	Medium	cli/cdk.py	urlopen može prihvatiti neočekivane URL šeme	Delimično validno; CLI treba ograničiti server URL na http/https
B404	Low	sandbox/docker_runner.py	Import subprocess	Očekivano, jer runner mora pokretati Docker
B108	Medium	sandbox/docker_runner.py	Hardkodovan /tmp u Docker tmpfs argumentu	Lažno pozitivan u kontekstu Docker CLI konfiguracije; ne koristi host temp direktorijum

ID	Severity	Fajl	Opis	Procena
B603	Low	sandbox/ docker_runner.py	subprocess.run sa listom argumenata	Prihvatljivo; ne koristi shell=True, argumenti su lista
B607	Low	sandbox/ docker_runner.py	Pokretanje programa preko parcijalne putanje docker	Može se ublažiti korišćenjem apsolutne putanje do Docker CLI-ja

7.3 Najvažniji nalaz: Flask debug mode

U backend/app.py:

```
app.run(host=host, port=port, debug=True)
```

Ovo je u redu za lokalni razvoj, ali nije prihvatljivo za produkciju. Flask debugger može prikazati detaljne stack trace-ove i u određenim uslovima omogućiti opasnu interakciju sa aplikacijom.

Predloženo rešenje:

```
debug = os.environ.get("FLASK_DEBUG", "0") == "1"
app.run(host=host, port=port, debug=debug)
```

Za produkciju treba koristiti WSGI server i reverse proxy.

7.4 Nalazi kada se uključe testovi

Pokrenuta je i šira komanda:

```
bandit -r backend sandbox cli tests -x backend/storage,sandbox/logs
```

Rezultat:

```
Total lines of code: 864
Total issues by severity:
  High: 1
  Medium: 2
  Low: 13
```

Dodatni nalazi dolaze iz testova:

- B101 `assert_used` u `tests/smoke_test.py`,
- B105 `hardcoded_password_string` za test lozinku `password123`,
- B605/B607 u `sandbox/test_scripts/malicious/os_system.py`.

Ovi nalazi su očekivani za test fajlove. `os_system.py` je namerno maliciozan test primer i služi da proverí da statička analiza blokira `import os`.

8. Testni primeri

Testni primeri postoje u `sandbox/test_scripts/` i `tests/smoke_test.py`.

8.1 Benigni primer

Fajl:

```
sandbox/test_scripts/benign/hello.py
```

Kod:

```
print("Hello from Oblak sandbox!")
print("Successful execution test.")

result = sum([1, 2, 3, 4, 5])
print(f"Sum: {result}")
```

Očekivani rezultat:

- `success = true`,

- `exit_code = 0,`
- `sandbox_execution_success.`

Stvarni rezultat testiranja:

```
Hello from Oblak sandbox!  
Successful execution test.  
Sum: 15
```

8.2 Maliciozni primer: OS pristup

Fajl:

```
sandbox/test_scripts/malicious/os_system.py
```

Kod:

```
import os  
  
os.system("ls -la /")
```

Očekivani rezultat:

- blokirano statičkom analizom,
- `blocked_by_policy = true,`
- `razlog: Forbidden module import: 'os'.`

8.3 Maliciozni primer: mrežni pristup

Fajl:

```
sandbox/test_scripts/malicious/socket_test.py
```

Kod:

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("google.com", 80))
```

Očekivani rezultat:

- blokirano statičkom analizom,
- razlog: `Forbidden module import: 'socket'.`

Čak i ako bi statička analiza bila zaobiđena, Docker se pokreće sa `--network none`, pa kontejner ne bi imao mrežni pristup.

8.4 Maliciozni primer: beskonačna petlja

Fajl:

```
sandbox/test_scripts/malicious/infinite_loop.py
```

Kod:

```
while True:
    pass
```

Očekivani rezultat:

- statička analiza ne blokira kod,
- Docker sandbox pokreće kod,
- `subprocess.run` prekida izvršavanje nakon 5 sekundi,
- `timeout = true`,
- `exit_code = 124`,
- `sandbox_execution_timeout`.

8.5 Test zavisnosti preko `requirements.txt`

Testiran je i primer sa `pyfiglet` bibliotekom:

```
import pyfiglet
print(pyfiglet.figlet_format("Oblak"))
```

Uz `requirements.txt`:

```
pyfiglet==1.0.2
```

Ovaj test potvrđuje da sistem:

1. čuva `requirements.txt`,
2. gradi Docker image sa instaliranim zavisnostima,
3. pokreće funkciju u tom image-u,
4. vraća rezultat korisniku.

8.6 Smoke test

Automatizovani test:

```
python tests\smoke_test.py
```

Test pokriva:

- registraciju korisnika,
- upload benignog koda,
- izvršavanje funkcije,
- upload malicioznog koda,
- odbijanje malicioznog koda statičkom analizom.

Test je uspešno prošao.

9. Zaključak

Oblak implementira osnovne funkcionalnosti serverless platforme za Python kod:

- CDK CLI autentifikaciju,
- upload `.py` fajla i opcionog `requirements.txt`,
- statičku analizu prenetog koda,
- pripremu Docker image-a sa zavisnostima,
- generisanje `run_url`,
- izolovano izvršavanje u Docker sandboxu,
- audit log u JSONL formatu,
- benigne i maliciozne test primere.

Najvažniji implementirani bezbednosni mehanizmi su:

- PBKDF2 hešovanje lozinki,
- Bearer token autentifikacija,
- AST statička analiza,
- ponovna analiza neposredno pre izvršavanja,
- Docker sandbox bez mreže,
- read-only fajl sistem,
- CPU/memory/PID limiti,
- timeout,
- centralizovan audit log.

Sistem je adekvatan za fakultetski projekat i jasno demonstrira princip odbrane u dubinu. Za produkcionu nivo bezbednosti ostaju otvorene stavke: Firecracker MicroVM, HTTPS, rate limiting, autentifikacija `/run` rute, hešovanje API tokena, dependency scanning, kriptografski zaštićeni audit logovi i dodatni Docker hardening pomoću seccomp/AppArmor profila.